# ML-based Deep Bitstream Inspection for Serverless FPGAs

SANTIAGO DE HEREDIA, Systems Group, ETH Zurich, Switzerland
MAXIMILIAN J. HEER, Systems Group, ETH Zurich, Switzerland
GUSTAVO ALONSO, Systems Group, ETH Zurich, Switzerland

Cloud and serverless FPGA platforms expose reconfigurable fabric to user-submitted designs, creating risks from malicious circuits such as ring oscillators. Existing machine-learning approaches typically inspect bitstreams offline in software. We instead study an on-FPGA checker that analyzes incoming partial bitstreams before reconfiguration. We generate benign Coyote and ring-oscillator bitstreams, convert them into fixed-size byte-image tensors, train a quantized CNN, and synthesize it using hls4ml. Our selected $256^2$-input, seven-layer model achieves 88.5% F1 and 95.1% ROC AUC. Its estimated preprocessing and inference latency is 3.35 ms, while the routed checker uses 4.47% of LUTs, 17.86% of BRAM, 1.15% of URAM, and 14.72% of DSPs on an Alveo U55C. These results demonstrate the feasibility of an on-FPGA checker for detecting malicious bitstreams before partial reconfiguration.

CCS Concepts: • **Security and privacy** → **Systems security**; • **Computer systems organization** → *Cloud computing*; • **Computing methodologies** → *Machine learning*; Neural networks.

Additional Key Words and Phrases: cloud FPGA security, bitstream inspection, partial bitsreams, malicious circuit detection, serverless FPGAs, partial reconfiguration, convolutional neural networks, ML for systems, hls4ml

**ACM Reference Format:**
Santiago de Heredia, Maximilian J. Heer, and Gustavo Alonso. 2026. ML-based Deep Bitstream Inspection for Serverless FPGAs. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (Conference acronym 'XX)*. ACM, New York, NY, USA, 13 pages. https://doi.org/XXXXXXX.XXXXXXX

## 1 Introduction

Field-programmable gate arrays (FPGAs) are increasingly used as datacenter accelerators because they combine programmability with high performance and energy efficiency [2, 4]. Cloud and serverless FPGA platforms extend this model by allowing users to deploy hardware applications on shared infrastructure. Partial reconfiguration (PR) makes this practical by loading user logic into a reconfigurable region while the trusted FPGA shell remains active [9, 15].

This deployment model exposes shared FPGA resources to untrusted user logic. Malicious circuits such as ring-oscillator (RO) arrays can generate sustained switching activity, causing voltage drops, timing faults, and denial of service [3, 8, 10, 14]. Demonstrated attacks on cloud FPGA infrastructure therefore motivate inspecting user bitstreams before partial reconfiguration [10].

Prior work has shown that machine-learning models can identify malicious designs from bitstream-derived representations without fully reconstructing their circuit semantics [5, 7, 13, 16]. However,

Authors' Contact Information: Santiago de Heredia, sdeheredia@ethz.ch, Systems Group, ETH Zurich, Zurich, Switzerland; Maximilian J. Heer, maximilian.heer@inf.ethz.ch, Systems Group, ETH Zurich, Zurich, Switzerland; Gustavo Alonso, alonso@inf.ethz.ch, Systems Group, ETH Zurich, Zurich, Switzerland.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

*Conference acronym 'XX, Woodstock, NY*

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-1-4503-XXXX-X/2018/06
https://doi.org/XXXXXXX.XXXXXXX

these approaches are commonly evaluated as offline software classifiers. A serverless FPGA platform instead needs a checker that can operate within the deployment path, retain useful detection quality, fit alongside platform and user logic, and avoid dominating PR latency.

We investigate an *on-FPGA checker* placed before partial reconfiguration. The checker transforms the raw bytes of an incoming partial bitstream into a fixed-size tensor, executes a quantized convolutional neural network (CNN), and returns a suspiciousness score to the deployment logic. We study this design in the context of Coyote, a datacenter-grade FPGA shell supporting PR and multitenancy [15]. We generate benign Coyote application bitstreams and RO-based malicious bitstreams, train compact CNNs, and synthesize selected models using hls4ml [1, 6].

Our evaluation identifies a seven-layer CNN operating on a $256 \times 256$ byte-image representation as the preferred deployment point. The resulting checker provides useful detection at millisecond-scale latency while leaving sufficient resources for the Coyote shell and user logic. The broader design-space exploration shows that modest improvements in software classification quality can require sharply higher on-chip memory and latency, making larger models unsuitable for deployment.

This work makes three contributions:

- We construct a dataset of benign Coyote partial bitstreams and RO-based malicious bitstreams across multiple floorplans and payload footprints.
- We develop an on-FPGA inspection pipeline combining deterministic byte-stream preprocessing, quantized CNN inference, and hls4ml synthesis.
- We evaluate detection quality, latency, and routed FPGA resource overhead together, identifying a practical deployment point for pre-reconfiguration inspection.

## 2 Background, Threat Model, and Related Work

### 2.1 Deployment Setting and Threat Model

Serverless FPGA platforms allow users to deploy hardware accelerators on shared reconfigurable infrastructure. Partial reconfiguration (PR) supports this model by dividing the FPGA into a trusted static shell and one or more reconfigurable regions containing user logic. The shell remains active while individual user regions are reconfigured, allowing the platform to retain control over host communication, memory, networking, and runtime services [9, 15].

This separation creates a security boundary. A user-submitted partial bitstream is an untrusted input that directly determines the circuit instantiated inside a reconfigurable region. Malicious circuits can exploit physical resources shared across the device, including power delivery, thermal headroom, and timing margins. We focus on ring-oscillator (RO) payloads, which produce sustained switching activity and can induce voltage drops, timing faults, or denial of service when instantiated at sufficient scale [3, 8, 10, 14].

We consider an attacker who can submit a syntactically valid partial bitstream for a user region. The bitstream contains an RO-based payload intended to pass existing platform checks and stress shared physical resources after configuration.

The defender places an on-FPGA checker before PR. The checker is intended to inspect the same buffered bitstream bytes that would subsequently be provided to the PR controller. It produces a maliciousness score that the trusted deployment logic converts into an allow or reject decision. A true positive blocks and logs a suspicious bitstream. A false negative allows a malicious bitstream to be configured, while a false positive rejects a benign accelerator.

Table 1. Comparison of malicious-bitstream detection approaches across deployment and implementation dimensions.

| Approach | Detection basis | Deployment | | Properties | | Target |
|---|---|---|---|---|---|---|
| | | Deployment integrated | On-FPGA | No reverse engineering | Datacenter target | |
| Platform checks | Provider-defined rules | ☒ | × | ☒ | ☒ | Cloud deployment |
| Structural scanning | Circuit structures | × | × | × | × | Device-specific FPGA |
| Offline ML | Learned features | × | × | ☒ | × | Host software |
| Embedded ML | Byte-frequency features | × | × | ☒ | × | PYNQ-Z1 |
| **This work** | **Learned byte patterns** | **Proposed** | ☒ | ☒ | ☒ | **Coyote/U55C** |

*Legend:* ☒indicates that a property is demonstrated, while ×indicates that it is not demonstrated. "Proposed" denotes integration designed as part of this work but not yet connected to the complete Coyote control path.

## 2.2 Existing Defenses and Research Gap

Existing defenses against malicious FPGA designs broadly fall into three groups. Table 1 summarizes their relationship to this work.

Platform and vendor checks are already part of practical cloud-FPGA deployment flows, but attacks have shown that syntactically valid power-wasting circuits can pass such checks [10]. Structural approaches instead search for specific circuit patterns, including self-oscillators [11, 19]. These methods provide direct evidence of suspicious structures, but often depend on device-specific architectural knowledge or bitstream reverse engineering.

Machine-learning approaches avoid explicit reconstruction of the implemented circuit. Prior work has classified malicious or Trojan-containing designs using features derived from FPGA bitstreams, including artificial neural networks, CNNs, and other supervised classifiers [5, 7, 13, 16]. These studies establish that bitstream-level representations contain learnable information. However, they primarily evaluate detection as an offline software task and do not jointly consider placement in a datacenter FPGA deployment path, FPGA resource cost, and inspection latency.

Stahle-Smith and Karakchi moved toward deployment-oriented inspection by extracting byte-frequency features and executing a Random Forest classifier on a PYNQ-Z1 embedded platform [18]. Their subsequent BLADEI framework extends this direction with a hybrid 1D-CNN and statistical feature model for deployment-time screening of raw bitstreams [17]. BLADEI identifies software preprocessing as the dominant latency bottleneck and evaluates a streaming byte-statistics engine in programmable logic, while classification remains on the embedded ARM processor. Our work instead applies deterministic byte-image preprocessing to Coyote partial bitstreams, synthesizes the quantized CNN itself into FPGA fabric using hls4ml, and targets an Alveo U55C path in which inspection occurs before PR.

This leaves three feasibility questions:

(1) **Can the checker retain useful detection quality, particularly a sufficiently low false-negative rate?**
(2) **Can the checker fit alongside the Coyote shell and user logic while adding acceptable inspection latency?**

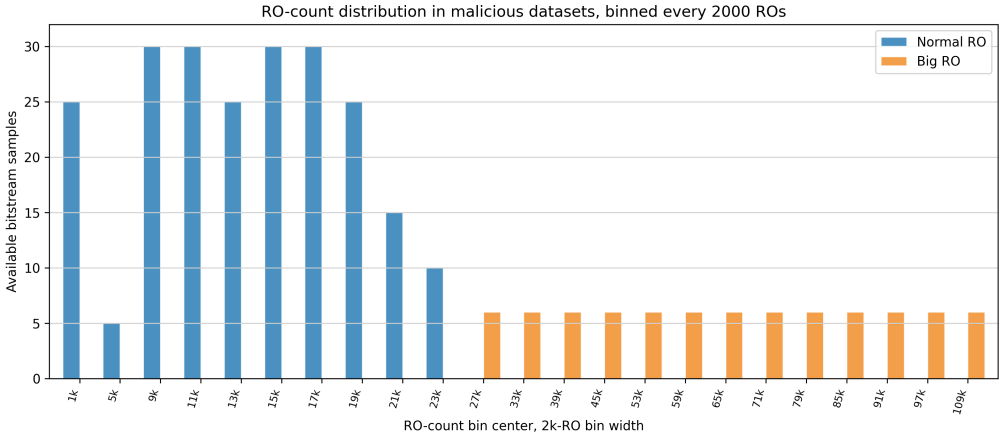



Fig. 1. Distribution of RO counts in the normal-RO dataset and the big-RO extension. Bars count available bitstream samples in 2k-RO bins.

(3) **Can preprocessing, inference, and decision logic be placed on the PR path without a separate host-side classification stage?**

The following sections evaluate these questions through dataset construction, model training, hardware synthesis, and a proposed Coyote integration.

## 3 Dataset and Preprocessing

### 3.1 Dataset

We construct a binary classification dataset of benign Coyote partial bitstreams and malicious RO-based partial bitstreams generated through the Coyote partial-reconfiguration flow. The benign class contains eight Coyote applications spanning HLS, cryptographic, communication, and compute examples. We build these applications across multiple Coyote floorplans, yielding 221 benign partial bitstreams.

Malicious samples use the RO-based designs of Mahmoud et al. [12], instantiated at different payload sizes and floorplans. The main corpus contains 225 malicious partial bitstreams. We additionally generate 78 higher-density samples, excluded from training and used only for the RO-footprint evaluation. Figure 1 shows the RO-count distribution of the main malicious dataset and its higher-density extension.

### 3.2 Preprocessing

We map each raw partial bitstream to a fixed-size $H \times W \times 1$ tensor using Algorithm 1. The transformation uniformly samples or pads the byte stream to $H \cdot W$ bytes, then inverts, normalizes, and reshapes the selected values.

Our final checker uses a $256 \times 256$ input, corresponding to 65,536 sampled bytes per partial bitstream. We evaluate additional input resolutions during the model design-space exploration in Section 4.

## 4 ML-Based Bitstream Inspection

We use convolutional neural networks (CNNs) to classify the fixed-size bitstream representations introduced in Section 3. Prior work has shown that bitstream-derived representations contain

**Algorithm 1** Deterministic bitstream preprocessing.

**Require:** Bitstream bytes $b$, target dimensions $H, W$

$T \leftarrow H \cdot W$
**if** $|b| > T$ **then**
  $i \leftarrow \lfloor \text{linspace}(0, |b| - 1, T) \rfloor$
  $b \leftarrow b[i]$
**else**
  $b \leftarrow \text{ZeroPad}(b, T)$
**end if**
$x \leftarrow \text{Reshape}((255 - b)/255, H, W, 1)$
**return** $x$

Table 2. Seven-stage quantized CNN architecture used by the on-FPGA checker. All convolutional stages use quantized ReLU activations and W8A8 quantization.

| Stage | Filters | Kernel | Stride | Pooling |
|---|---|---|---|---|
| Conv 0 | 8 | $5 \times 5$ | $2 \times 2$ | $2 \times 2$ |
| Conv 1 | 16 | $3 \times 3$ | $1 \times 1$ | $2 \times 2$ |
| Conv 2 | 24 | $3 \times 3$ | $1 \times 1$ | $2 \times 2$ |
| Conv 3 | 24 | $3 \times 3$ | $1 \times 1$ | $2 \times 2$ |
| Conv 4 | 32 | $3 \times 3$ | $1 \times 1$ | $2 \times 2$ |
| Conv 5 | 32 | $3 \times 3$ | $1 \times 1$ | $2 \times 2$ |
| Conv 6 | 32 | $3 \times 3$ | $1 \times 1$ | $2 \times 2$ |
| Output | 1 | – | – | AvgPool |

learnable structure and can be used to distinguish benign and malicious FPGA designs [5, 7, 13, 16]. CNNs are also well suited to our deployment goal because quantized models can be translated into FPGA circuits using tools such as hls4ml [1, 6].

### 4.1 CNN Architecture

The selected checker uses a seven-stage convolutional architecture operating on a $256 \times 256$ byte-image tensor. Each stage consists of a quantized convolution, quantized ReLU activation, and max pooling. Channel width increases from 8 filters in the first stage to 32 filters in the later stages. A final average-pooling operation reduces the remaining feature map before a single quantized dense output produces the classification logit.

Table 2 summarizes the architecture. We use 8-bit weights and activations (W8A8) and do not use batch normalization. The output logit is converted to a maliciousness probability using a sigmoid; the decision threshold is applied later by the deployment logic.

### 4.2 Training

Models are implemented in TensorFlow [**?** ] using QKeras [**?** ]. We use quantization-aware training [**?** ], so that the model is trained under the same fixed-precision behavior used during hardware generation. Training minimizes binary cross-entropy from logits. We augment the training data with horizontal and vertical flips and use a balanced production training set containing equal numbers of benign and malicious samples.

During model development, we use stratified cross-validation to compare model configurations [**?** ]. After selecting the final architecture, we train a production model separately and evaluate it on the held-out test set described in Section 6.1.

### 4.3 Hardware-Aware Design-Space Exploration

Model selection considers hardware feasibility in addition to software classification quality. Rather than exhaustively evaluating every combination of parameters, we explore the design space in stages. We first vary input resolution and CNN depth to determine how model capacity and retained bitstream information affect classification quality. We then evaluate quantization and pruning as model-compression techniques before passing selected networks to the hardware synthesis flow described in Section 5.

The principal model-level dimensions are therefore input resolution, network depth, and numerical precision. Hardware parallelism is explored separately after conversion to hls4ml by varying operation reuse and per-layer configuration.

The resulting design-space exploration selects the $256 \times 256$, seven-layer W8A8 CNN as the on-FPGA checker used throughout the remainder of the paper. We also synthesize the same architecture at $512 \times 512$ to evaluate how hardware cost scales with input resolution.

## 5 On-FPGA Checker

We next map the trained bitstream classifier into FPGA logic and consider how the resulting checker can be placed in the Coyote deployment path. The hardware design consists of two components: deterministic byte-stream preprocessing and the quantized CNN classifier. Importantly, both execute on the FPGA, allowing inspection to take place without a separate host-side classification stage.

### 5.1 Hardware Generation

We convert the trained QKeras CNN to hardware using hls4ml [1, 6]. hls4ml translates the quantized network into an HLS implementation, which is first evaluated through C simulation and subsequently synthesized to RTL. The final models use quantization-aware training, so the weight and activation precision used during training already reflects the W8A8 representation targeted by the generated hardware.

Before synthesis, we compare the hls4ml C-simulation output against the corresponding QKeras model. We require matching classifications on the validation inputs and inspect output deviations before synthesis. This provides a parity check between the trained model and its fixed-point hardware implementation.

The preprocessing operation from Section 3.2 is implemented separately in HLS C++ and lowered to RTL. It operates directly on the incoming bitstream bytes and reproduces the deterministic sampling, padding, inversion, and normalization used during training. The latency reported in Section 6.4 therefore includes both FPGA-side preprocessing and CNN inference.

### 5.2 Hardware Optimization

Once the trained network is fixed, hls4ml exposes a hardware design space that trades resource usage against inference latency. The main parameter is the *reuse factor*, which controls how aggressively arithmetic units are shared across operations. Lower reuse increases parallelism and reduces latency at the cost of additional hardware, while higher reuse reduces resource use but increases execution time.

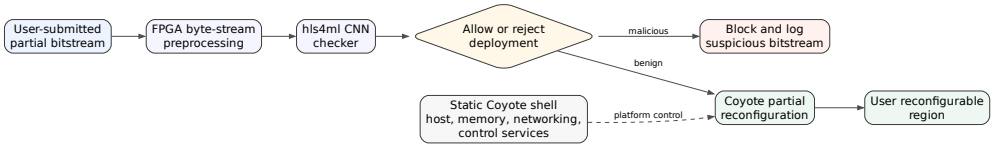



Fig. 2. Proposed integration of the on-FPGA checker into the Coyote deployment path. FPGA-side preprocessing and CNN inference inspect an incoming partial bitstream before it is passed to the partial-reconfiguration controller.

We initially evaluate resource-oriented hls4ml configurations and sweep reuse factors for the selected CNN architectures. A single global reuse setting, however, provides only a coarse trade-off because convolutional stages differ substantially in their operation counts and hardware cost. We therefore also apply manual per-layer reuse settings, assigning greater parallelism to latency-dominant layers while retaining more sharing elsewhere.

Quantization and pruning are additionally explored as hardware-aware model transformations. The final checker uses W8A8 quantization trained through QKeras quantization-aware training. For each transformed model, we retain a configuration only when software detection quality and hls4ml prediction parity remain acceptable. Section 6.4 evaluates the resulting implementations jointly in terms of detection quality, latency, and routed FPGA resources.

### 5.3 Proposed Coyote Integration

Coyote separates trusted platform services from partially reconfigurable user regions, providing a natural location for bitstream inspection before PR [15]. We propose placing the on-FPGA checker in this trusted deployment path, as illustrated in Figure 2.

An incoming user partial bitstream is first processed by the FPGA-side preprocessing logic and passed to the synthesized CNN. The CNN produces a maliciousness score, which trusted deployment logic compares against a configured threshold. Bitstreams classified as suspicious are rejected, while accepted bitstreams are allowed to proceed to the Coyote partial-reconfiguration controller.

The integration shown in Figure 2 is a proposed deployment architecture rather than a completed end-to-end Coyote implementation. This work implements and evaluates the FPGA preprocessing and CNN checker components; connection to Coyote's PR controller and the associated control-plane logic remain future work.

## 6 Evaluation

We evaluate whether the proposed on-FPGA checker provides useful detection quality while remaining practical for deployment alongside the Coyote shell and user logic. In particular, we answer four questions:

(1) **Detection quality.** How accurately does the checker distinguish benign Coyote partial bitstreams from bitstreams containing ring-oscillator (RO) payloads?
(2) **RO-footprint sensitivity.** How does detection change as the malicious RO footprint increases?
(3) **Hardware feasibility.** What latency and resource overhead does the checker introduce on the target FPGA?
(4) **Deployment-path overhead.** How significant is the added inspection latency relative to partial reconfiguration?

Table 3. Detection quality of the selected $256^2$-input, seven-layer checker on the held-out test set.

| Accuracy | F1 | TPR | FNR | FPR | ROC AUC |
|---|---|---|---|---|---|
| 87.7% | 88.5% | 92.0% | 8.0% | 16.9% | 95.1% |

Our evaluation considers both the $256^2$- and $512^2$-input, seven-layer CNNs to characterize the tradeoff between detection quality and hardware cost. Retaining more bitstream information can improve classification quality, but also increases latency and on-chip resource requirements.

### 6.1 Experimental Setup

We evaluate the checker on the held-out test set defined in Section 4. The positive class is the malicious RO class: a false negative therefore represents a malicious bitstream accepted by the checker, while a false positive represents a benign accelerator rejected by the platform. The separate high-density RO set introduced in Section 3 is evaluated only in the RO-footprint analysis.

Predictions use a probability threshold of 0.5, with higher values indicating greater confidence that a bitstream is malicious. We report accuracy, F1, true-positive rate (TPR), false-negative rate (FNR), false-positive rate (FPR), and ROC AUC.

Hardware results target the AMD Alveo U55C. Resource percentages are attributed to the routed checker hierarchy relative to the complete device. As described in Section 5, the reported checker latency combines the FPGA byte-stream preprocessing latency and hls4ml inference latency. It excludes host-side setup, memory transfers, output readback, logging, and control-plane operations. Complete integration with Coyote's partial-reconfiguration controller remains future work.

### 6.2 Detection Quality

Table 3 reports the detection quality of the selected checker on the held-out test set. The model reaches 87.7% accuracy and an F1 score of 88.5%. It detects 92.0% of the malicious bitstreams, corresponding to an FNR of 8.0%. Its ROC AUC of 95.1% indicates that the model provides substantial separation between the benign and malicious classes across possible decision thresholds.

The security-relevant error is the FNR because each false negative represents an malicious bitstream that would pass the checker. At the default threshold, the model misses 8.0% of malicious test samples. The corresponding FPR is 16.9%, meaning that the same operating point rejects a non-negligible fraction of benign applications.

This trade-off depends on the decision threshold. Reducing the threshold would reject more malicious bitstreams, but would also block more benign accelerators. Increasing it would reduce the operational cost of false positives while admitting more malicious designs. We use threshold 0.5 as a fixed and reproducible threshold throughout the evaluation. A production deployment could instead select the threshold according to an explicit platform policy, such as minimizing the FNR subject to a maximum acceptable FPR.

### 6.3 Sensitivity to RO Footprint

Aggregate test-set metrics do not show whether all malicious designs are equally difficult to detect. We therefore evaluate the checker as a function of the estimated FPGA footprint occupied by the RO payload. This analysis combines the standard malicious test samples with the separate high-density RO extension.

Figure 3 reports the detection rate across RO LUT share. Most false negatives occur among the smallest evaluated RO footprints. As the amount of RO logic increases, the detection rate rises and

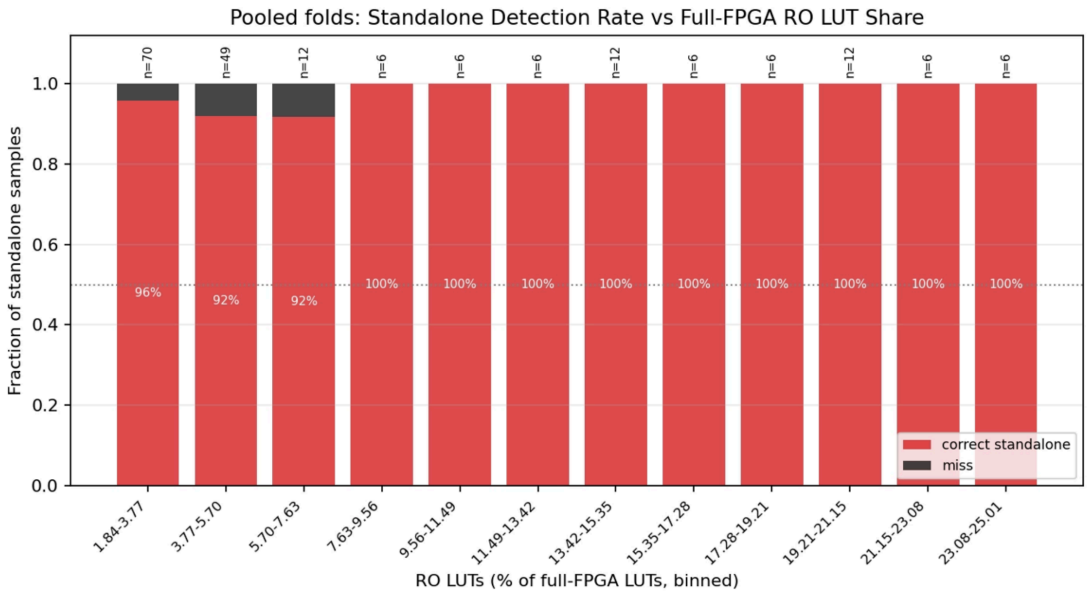



Fig. 3. Detection rate of the selected checker as a function of estimated RO LUT share. Misses concentrate among the smallest malicious footprints, while larger evaluated RO payloads are detected reliably.

remains close to one across the larger footprint ranges. On the high-density extension, the selected checker reaches a TPR of 98.7%.

This result aligns with demonstrated RO-based attacks, which target availability through power-hammering and denial of service, or integrity through voltage-induced fault injection. FPGADefender showed that an optimized design exceeded the Ultra96 board's power budget using about 3% of its LUTs and crashed the board immediately at 5.2% LUT utilization [11]. FPGAhammer induced faults using attacker designs occupying 30–50% of the available LUTs [**?** ], and Mahmoud et al. observed targeted AES faults using tens of thousands of ROs on a cloud-scale FPGA [12]. The checker is therefore strongest in the footprint range relevant to these demonstrated attacks, while smaller RO payloads for side-channel attacks remain more difficult to detect.

### 6.4 Hardware Evaluation

We convert the selected W8A8 QKeras model using hls4ml. The generated model is first evaluated through C simulation to confirm consistency with the quantized software model. We then synthesize the checker and place and route its wrapper for the Alveo U55C. Manual per-layer reuse settings are used to reduce latency without adopting the resource cost of a globally latency-optimized implementation.

Table 4 reports the routed resource attribution and combined preprocessing-plus-inference latency. The selected checker adds 4.47% of device LUTs, 17.86% of BRAM, 1.15% of URAM, and 14.72% of DSPs. Its estimated latency is 3.35 ms per partial bitstream.

The selected design fits alongside the baseline shell, although its BRAM and DSP requirements are substantial. LUT use remains moderate, while BRAM is the most important resource constraint. After adding the checker, memory resources must still remain available for platform services and

Table 4. Routed resource attribution and latency on the Alveo U55C. Checker entries are additions to the baseline Coyote shell. The $512^2$ implementation is included only as a higher-resolution scaling point.

| Design | LUT | BRAM | URAM | DSP | Latency |
|---|---|---|---|---|---|
| Baseline Coyote | 10.26% | 9.77% | 0% | 0% | – |
| Added $256^2$ checker | +4.47% | +17.86% | +1.15% | +14.72% | 3.35 ms |
| Added $512^2$ checker | +4.90% | +61.78% | +1.77% | +14.45% | 6.53 ms |

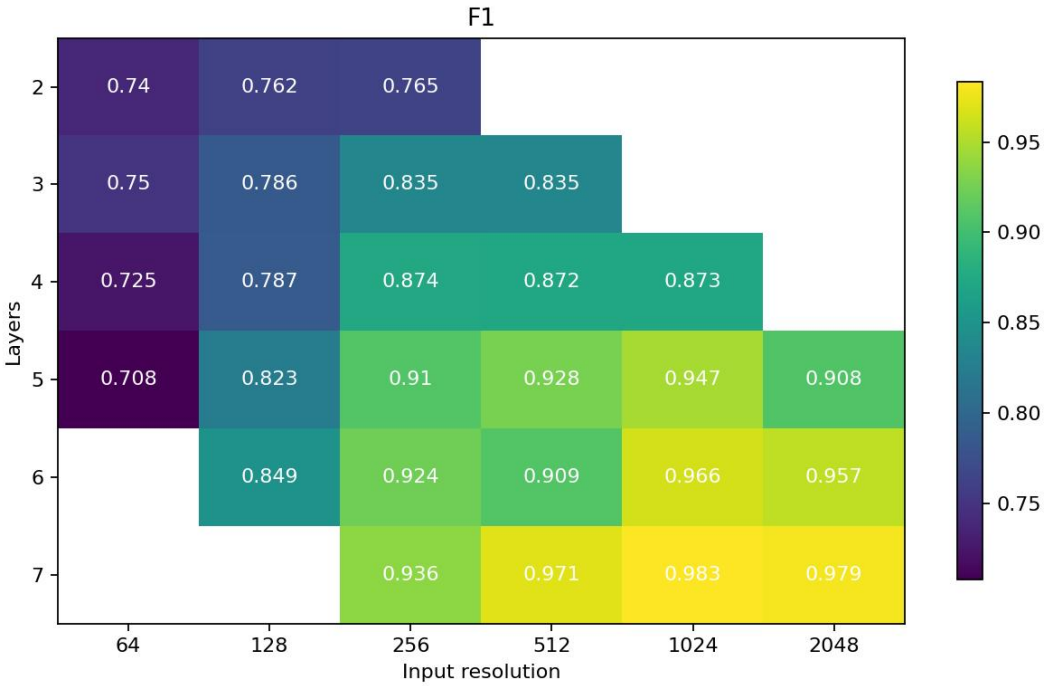



Fig. 4. Pooled-validation F1 across input resolution and CNN depth. Detection quality generally improves with larger inputs and deeper models.

user accelerators. This makes the memory footprint more significant than the comparatively small LUT increase.

Figure 4 summarizes the software design space that motivates the hardware comparison. Detection quality generally improves as the input resolution and CNN depth increase. The seven-layer models achieve the strongest pooled-validation results, while the gains from increasing resolution become smaller at the largest configurations. These results motivate evaluating whether the additional detection quality of larger models justifies their hardware cost. The values in this figure are pooled-validation results used for model selection and are therefore distinct from the held-out test results reported in Table 3.

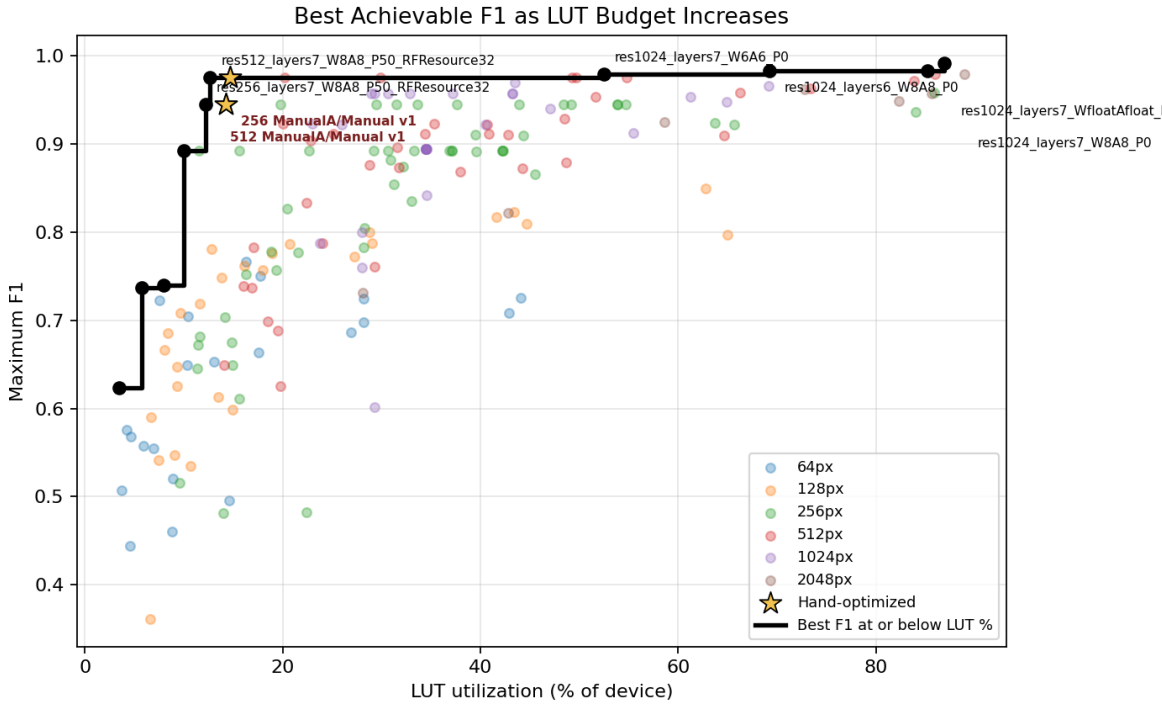



Fig. 5. F1 versus LUT utilization across synthesized candidates. The black curve shows the best-achievable F1 frontier, while stars mark the hand-optimized $256^2$ and $512^2$ designs.

To examine scaling, we also synthesize the same seven-layer architecture with a $512 \times 512$ input. This configuration retains four times as many sampled bytes as the selected checker. It improves F1 from 88.5% to 90.5%, but nearly doubles the checker latency from 3.35 ms to 6.53 ms. More importantly, its attributed BRAM use rises from 17.86% to 61.78% of the complete device.

The higher-resolution result demonstrates that model scaling quickly becomes impractical. The small improvement in classification quality requires more than three times the BRAM of the selected checker and leaves limited memory capacity for the shell, platform functionality, and user logic. We therefore treat the $512^2$ implementation as evidence of the hardware scaling limit rather than as a deployment candidate.

Figure 5 places the selected checker in the broader hardware design space. Increasing input resolution and reducing operation reuse can improve detection quality or latency in isolation, but moves the design toward higher resource consumption. Per-layer tuning shifts the selected $256^2$ checker toward a more useful operating point. The resulting model retains useful detection quality while avoiding the rapid BRAM growth of the higher-resolution implementation.

### 6.5 Deployment-Path Overhead

Inspection is performed before partial reconfiguration and therefore adds to the deployment latency of each submitted bitstream. Table 5 compares the selected checker's 3.35 ms latency with previously reported Coyote partial-reconfiguration measurements.

Table 5. Estimated overhead of the selected checker relative to previously reported Coyote partial-reconfiguration latency [9].

| PR region | PR latency | Checker | Overhead |
|---|---|---|---|
| 6% | 6.04 ms | 3.35 ms | 55.5% |
| 33% | 36.94 ms | 3.35 ms | 9.1% |
| 66% | 73.27 ms | 3.35 ms | 4.6% |

For a small 6% reconfigurable region, inspection adds more than half of the reference PR latency. The relative cost falls to 9.1% for a 33% region and 4.6% for a 66% region. The checker is therefore most attractive for deployments in which the reconfiguration itself already takes tens of milliseconds. Its cost is more pronounced for small regions with short reconfiguration times.

These comparisons provide deployment context rather than an end-to-end measurement of an integrated system. The reported 3.35 ms covers FPGA-side preprocessing and inference but excludes command handling, buffering, logging, threshold-policy enforcement, and transfer of an accepted bitstream to the PR controller. Full integration may therefore add further fixed overhead.

Overall, the $256^2$ seven-layer CNN is the selected design point. It reaches 88.5% F1 with an 8.0% false-negative rate, detects 98.7% of the separate high-density RO samples, and completes preprocessing and inference in an estimated 3.35 ms. Its resource footprint is compatible with feasibility on the U55C, while the higher-resolution experiment shows that increasing model input size rapidly becomes constrained by BRAM and latency. These results support the feasibility of an on-FPGA checker for RO-focused, defense-in-depth inspection before partial reconfiguration.

## 7 TODO: Discussion + Conclusion

## References

[1] Thea Aarrestad, Vladimir Loncar, Nicolò Ghielmetti, Maurizio Pierini, Sioni Summers, Jennifer Ngadiuba, Christoffer Petersson, Hampus Linander, Yutaro Iiyama, Giuseppe Di Guglielmo, Javier Duarte, Philip Harris, Dylan Rankin, Sergo Jindariani, Kevin Pedro, Nhan Tran, Mia Liu, Edward Kreinar, Zhenbin Wu, and Duc Hoang. 2021. Fast convolutional neural networks on FPGAs with hls4ml. *Machine Learning: Science and Technology* 2, 4 (2021), 045015. doi:10.1088/2632-2153/ac0ea1

[2] Christophe Bobda, Joel Mandebi Mbongue, Paul Chow, Mohammad Ewais, Naif Tarafdar, Juan Camilo Vega, Ken Eguro, Dirk Koch, Suranga Handagala, Miriam Leeser, Martin Herbordt, Hafsah Shahzad, Peter Hofste, Burkhard Ringlein, Jakub Szefer, Ahmed Sanaullah, and Russell Tessier. 2022. The Future of FPGA Acceleration in Datacenters and the Cloud. *ACM Transactions on Reconfigurable Technology and Systems* 15, 3 (2022), 1–42. doi:10.1145/3506713

[3] Andrew Boutros, Mathew Hall, Nicolas Papernot, and Vaughn Betz. 2020. Neighbors From Hell: Voltage Attacks Against Deep Learning Accelerators on Multi-Tenant FPGAs. In *2020 International Conference on Field-Programmable Technology (ICFPT)*. 103–111. doi:10.1109/ICFPT51103.2020.00023

[4] Adrian M. Caulfield, Eric S. Chung, Andrew Putnam, Hari Angepat, Jeremy Fowers, Michael Haselman, Stephen Heil, Matt Humphrey, Puneet Kaur, Joo-Young Kim, Daniel Lo, Todd Massengill, Kalin Ovtcharov, Michael Papamichael, Lisa Woods, Sitaram Lanka, Derek Chiou, and Doug Burger. 2016. A Cloud-Scale Acceleration Architecture. In *Proceedings of the 49th Annual IEEE/ACM International Symposium on Microarchitecture*. 1–13. doi:10.1109/MICRO.2016.7783710

[5] Jayeeta Chaudhuri and Krishnendu Chakrabarty. 2022. Detection of Malicious FPGA Bitstreams using CNN-Based Learning. In *2022 IEEE European Test Symposium (ETS)*. 1–2. doi:10.1109/ETS54262.2022.9810438

[6] Javier Duarte, Song Han, Philip Harris, Sergo Jindariani, Edward Kreinar, Benjamin Kreis, Jennifer Ngadiuba, Maurizio Pierini, Ryan Rivera, Nhan Tran, and Zhenbin Wu. 2018. Fast inference of deep neural networks in FPGAs for particle physics. *Journal of Instrumentation* 13, 07 (2018), P07027. doi:10.1088/1748-0221/13/07/P07027

[7] Rana Elnaggar, Jayeeta Chaudhuri, Ramesh Karri, and Krishnendu Chakrabarty. 2023. Learning Malicious Circuits in FPGA Bitstreams. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 42, 3 (2023), 726–739. doi:10.1109/TCAD.2022.3190771

[8] Dennis R. E. Gnad, Fabian Oboril, and Mehdi B. Tahoori. 2017. Voltage Drop-based Fault Attacks on FPGAs Using Valid Bitstreams. In *2017 27th International Conference on Field Programmable Logic and Applications (FPL)*. 1–7. doi:10.23919/FPL.2017.8056840

[9] Dario Korolija, Timothy Roscoe, and Gustavo Alonso. 2020. Do OS Abstractions Make Sense on FPGAs?. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*. USENIX Association, 991–1010.

[10] Tuan La, Khoa Dang Pham, Joseph Powell, and Dirk Koch. 2021. Denial-of-Service on FPGA-based Cloud Infrastructures: Attack and Defense. *IACR Transactions on Cryptographic Hardware and Embedded Systems* 2021, 3 (2021), 441–464. doi:10.46586/tches.v2021.i3.441-464

[11] Tuan Minh La, Kaspar Mätas, Nikola Grunchevski, Khoa Dang Pham, and Dirk Koch. 2020. FPGADefender: Malicious Self-Oscillator Scanning for Xilinx UltraScale+ FPGAs. *ACM Transactions on Reconfigurable Technology and Systems* 13, 3 (2020), 15:1–15:31. doi:10.1145/3402937

[12] Dina G Mahmoud, Ognjen Glamočanin, Francesco Regazzoni, and Mirjana Stojilović. 2023. Practical Implementations of Remote Power Side-Channel and Fault-Injection Attacks on Multitenant FPGAs. In *Security of FPGA-Accelerated Cloud Computing Environments*. Springer, 101–135.

[13] Alessandro Palumbo, Luca Cassano, Biagio Luzzi, Juan Antonio Hernández, Pedro Reviriego, Giuseppe Bianchi, and Marco Ottavi. 2022. Is your FPGA bitstream Hardware Trojan-free? Machine learning can provide an answer. *Journal of Systems Architecture* 128 (2022), 102543. doi:10.1016/j.sysarc.2022.102543

[14] George Provelengios, Daniel Holcomb, and Russell Tessier. 2020. Power Wasting Circuits for Cloud FPGA Attacks. In *2020 30th International Conference on Field-Programmable Logic and Applications (FPL)*.

[15] Benjamin Ramhorst, Dario Korolija, Maximilian Jakob Heer, Jonas Dann, Luhao Liu, and Gustavo Alonso. 2025. Coyote v2: Raising the Level of Abstraction for Data Center FPGAs. In *Proceedings of the ACM Symposium on Operating Systems Principles*. arXiv:2504.21538 [cs.AR] doi:10.1145/3731569.3764845

[16] Jens Rettkowski, Safdar Mahmood, Arij Shallufa, Michael Hübner, and Diana Göhringer. 2019. Inspection of Partial Bitstreams for FPGAs Using Artificial Neural Networks. In *2019 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW)*. 83–86. doi:10.1109/IPDPSW.2019.00023

[17] Rye Stahle-Smith, Carter Antley, Jason D. Bakos, and Rasha Karakchi. 2026. Hardware-Accelerated Line-Rate Bitstream Screening for Secure FPGA Reconfiguration. arXiv:2605.08984 [cs.CR] https://arxiv.org/abs/2605.08984

[18] Rye Stahle-Smith and Rasha Karakchi. 2025. Real-time ML-based Defense Against Malicious Payload in Reconfigurable Embedded Systems. arXiv:2509.02387 [cs.CR] https://arxiv.org/abs/2509.02387

[19] Junghwan Yoon, Yeseul Seo, Jihun Jang, Minsoo Cho, Jaeseung Kim, Hyung-Min Kim, and Taesoo Kwon. 2018. A Bitstream Reverse Engineering Tool for FPGA Hardware Trojan Detection. In *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*. 2318–2320. doi:10.1145/3243734.3278487

Received TODO; revised TODO; accepted TODO